Security

SeLinux Management

Introduction

- SeLinux, Security-Enhanced Linux, is an additional method to protect your system.
- It is a set of security rules that determine which process can access which files, directories, ports, etc. Every file, process, directory and port have special security label called SELinux contexts.
- A context is a name that is used by the SELinux policy to determine whether or not a process can access a file, directory or port.
- SELinux labels have several contexts, but the most important here is type context.
  - The type context name ends with _t

SELinux Modes

- Enforcing mode
  - SELinux both logs and protects
- Permissive mode
  - SELinux logs only
- Disabled mode
  - SELinux is completely disabled.

- System must be rebooted to disable SELinux or to get from disable mode to enforcing or permissive.

Display and Modify SELinux Modes

- Edit in `/etc/sysconfig/selinux` to change the default SELinux mode
  - `SELINUX=enforcing`
- To display the current SELinux
  - `getenforce` command
- To modify the current SELinux
  - `setenforce` command

Display SELinux File Contexts

- Many commands have -Z option to display or set SELinux context
  - `ps`, `ls`, `cp` and `mkdir` commands
- By default the context of the parent directory is assigned to the newly created file (`vi`, `cp`, `touch`)

Examples

```
ls -Zd /var/www/html
```

```
drwxr-xr-x. root root sytem_u:object_r:httpd_sys_content_t:s0
   /var/www/html
```

```
touch /var/www/html/index.html
```

```
ls -Z /var/www/html/index.html
```

```
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html
```

Modify SELinux File Contexts

- `semanage fcontext` can be used to display and modify the rules that `restorecon` uses to set default file contexts.

- `restorecon` is part of the `policycoreutil` package and `semanage` is part of the `policycoreutil-python` package.

Examples

```
touch /tmp/file1 /tmp/file2
ls -Z /tmp/file*
-rw-r--r--. root root unconfined_u:object_r:user_tmp_t:s0 /tmp/file1
-rw-r--r--. root root unconfined_u:object_r:user_tmp_t:s0 /tmp/file2
mv /tmp/file1 /var/www/html
cp /tmp/file2 /var/www/html
ls -Z /var/www/html/file*
-rw-r--r--. root root unconfined_u:object_r:user_tmp_t:s0 /var/www/html/file1
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/file2
```

Examples Cont'd

```
semanage fcontext -l
                    ….
/var/www(/.*)?      all files
     system_u:object_r:httpd_sys_content_t:s0
….
```

```
restorecon -Rv /var/www/
restorecon reset /var/www/html/file1 context unconfined_u:object_r:user_tmp_t:s0
   →system_u:object_r:httpd_sys_content_t:s0
```

```
ls -Z /var/www/html/file*
-rw-r--r--. root root system_u:object_r:httpd_sys_content_t:s0 /var/www/html/file1
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/file2
```

Examples Cont'd

```
mkdir /virtual ; touch /virtual/index.html
ls -Zd /virtual/
 drwxr-xr-x. root root unconfined_u:object_r:default_t:s0 /virtual/
ls -Z /virtual
-rw-r--r--. root root unconfined_u:object_r:default_t:s0 index.html
semanage fconext -a -t httpd_sys_content_t '/virtual'
restorecon -RFvv /virtual
ls -Zd /virtual
 drwxr-xr-x. root root unconfined_u:object_r:httpd_sys_content_t:s0 /virtual/
ls -Z /virtual
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 index.html
```

Managing SELinux Booleans

- SELinux booleans are switches that change the behavior of the SELinux policy.
- They are rules that can be enabled or disabled
- To display the booleans
  - Use `getsebool` command
- To modify the booleans
  - Use `setsebool` command
- To permanent modify a boolean
  - Use `setsebool -P` command
- To show whether a boolean is permanent or not
  - Use `semanage boolean -l` command

Examples

```
getsebool -a
….
allow_console_login - - > on
….
getsebool httpd_enable_homedirs
httpd_enable_homedirs - -> off
setsebool httpd_enable_homedirs on
semanage boolean -l | grep httpd_enable_homedirs
httpd_enable_homedirs      -> off   Allow httpd to read home directories
getsebool httpd_enable_homedirs
httpd_enable_homedirs - -> on
setsebool -P httpd_enable_homedirs on
semanage boolean -l | grep httpd_enable_homedirs
httpd_enable_homedirs      -> on   Allow httpd to read home directories
```

OpenSSH Server

Introduction to OpenSSH Server

- OpenSSH is one of the most powerful tools for administrating remote systems. It uses strong encryption and host keys as a protection against network sniffing.

- Traditional tools used to accomplish these functions, such as telnet or rcp, are insecure and transmit the user's password in clear text when used.

- It is the only network service which is enabled by default and is remotely accessible.

Example

- If a remote computer is connecting with the ssh client application, the OpenSSH server sets up a remote control session after authentication.

```
ssh -X user@hostname
```

- If a remote user connects to an OpenSSH server with `scp`, the OpenSSH server daemon initiates a secure copy of files between the server and client after authentication.

```
scp local-path  user@rhost:rpath
rsync  user@rhost:rpath local-path
```

- OpenSSH can use many authentication methods, including plain password, public key, and Kerberos tickets.

SSH Keys

- SSH keys allow authentication between two hosts without the need of a password.

- SSH key authentication uses two keys a private key and a public key.

- Public key encryption
- Encryption using a matched pair of encryption keys, a public key and a private key, Also called asymmetric encryption.

SSH Keys Cont'd

- Public key
- The encryption key given to anyone who wants to communicate confidentially with the owner of the private key
- What is encrypted by the public key can be decrypted by the private key
- Private key
- The encryption key kept secret by the owner
- What is encrypted by the private key can be decrypted by anyone with the matching public key

SSH Keys Cont'd

- To generate the keys

```
ssh-keygen -t rsa
```

- The public key is saved in the file `~/.ssh/id_rsa.pub`
- The private key is saved in the file `~/.ssh/id_rsa`

- Then copy the `id_rsa.pub` file to the remote host

```
ssh-copy-id -i .ssh/id_rsa.pub username@remotehost
```

Configuring OpenSSH

- OpenSSH configuration file is `/etc/ssh/sshd_config`

- There are many directives in the sshd configuration file controlling such things as communication settings and authentication modes.

- Prior to editing the configuration file, you should make a copy of the original file and protect it from writing so you will have the original settings as a reference and to reuse as necessary.

Configuring OpenSSH cont'd

```
# cp /etc/ssh/sshd_config /etc/ssh/sshd_config.original
# chmod a-w /etc/ssh/sshd_config.original
```

- To set your OpenSSH to listen on TCP port 2222 instead of the default TCP port 22

```
Port 2222
```

- To have `sshd` allow public key-based login credentials

```
PubkeyAuthentication yes
```

Configuring OpenSSH cont'd

- To make your OpenSSH server display the contents of the `/etc/issue.net` file as a pre-login banner

```
Banner /etc/issue.net
```

- To disable root access

```
PermitRootLogin no
```

- After making changes to the `/etc/ssh/sshd_config` file, restart the `sshd` server application to effect the changes

```
systemctl restart sshd
```

Thanks :)